STEM BUILDERS


# OOPs - Classes & Objects

# 9. OOPs – Creating Classes & Objects

So far you have learned about Python's core data types: strings, numbers, lists, tuples, and dictionaries. In this section you will learn about the last major data structure, classes. Classes are quite unlike the other data types, in that they are much more flexible. Classes allow you to define the information and behavior that characterize anything you want to model in your program. Classes are a rich topic, so you will learn just enough here to dive into the projects you'd like to get started on.

Before we understand how to create classes, we need to understand OOPs.

**What is OOPs?**

Python has been an object-oriented language since it existed. Because of this, creating and using classes and objects are downright easy. This chapter helps you become an expert in using Python's object-oriented programming support.

Here is small introduction of Object-Oriented Programming (OOP)

## Overview of OOP Terminology

- **Class**: A user-defined prototype for an object that defines a set of attributes that characterize any object of the class. The attributes are data members (class variables and instance variables) and methods, accessed via dot notation.
- **Class variable**: A variable that is shared by all instances of a class. Class variables are defined within a class but outside any of the class's methods. Class variables are not used as frequently as instance variables are.
- **Data member**: A class variable or instance variable that holds data associated with a class and its objects.
- **Function overloading**: The assignment of more than one behavior to a particular function. The operation performed varies by the types of objects or arguments involved.
- **Instance variable**: A variable that is defined inside a method and belongs only to the current instance of a class.
- **Inheritance**: The transfer of the characteristics of a class to other classes that are derived from it.
- **Instance:** An individual object of a certain class. An object obj that belongs to a class Circle, for example, is an instance of the class Circle.
- **Instantiation**: The creation of an instance of a class.
- **Method**: A special kind of function that is defined in a class definition.
- **Object**: A unique instance of a data structure that's defined by its class. An object comprises both data members (class variables and instance variables) and methods.
- **Operator overloading**: The assignment of more than one function to a particular operator.

# What are classes?

Classes are a way of combining information and behavior. For example, let's consider what you'd need to do if you were creating a rocket ship in a game, or in a physics simulation. One of the first things you'd want to track are the x and y coordinates of the rocket. Here is what a simple rocket ship class looks like in code:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
```

One of the first things you do with a class is to define the ***_init_*()** method. The __init__() method sets the values for any parameters that need to be defined when an object is first created. The *self* part will be explained later; basically, it's a syntax that allows you to access a variable from anywhere else in the class.

The Rocket class stores two pieces of information so far, but it can't do anything. The first behavior to define is a core behavior of a rocket: moving up. Here is what that might look like in code:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0

    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1
```

The Rocket class can now store some information, and it can do something. But this code has not actually created a rocket yet. Here is how you actually make a rocket:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
```

```
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a Rocket object.
my_rocket = Rocket()
print(my_rocket)
```

**Output:**

```
<__main__.Rocket object at 0x7f6f50c39190>
```

To actually use a class, you create a variable such as *my_rocket*. Then you set that equal to the name of the class, with an empty set of parentheses. Python creates an **object** from the class. An object is a single instance of the Rocket class; it has a copy of each of the class's variables, and it can do any action that is defined for the class. In this case, you can see that the variable my_rocket is a Rocket object from the __main__ program file, which is stored at a particular location in memory.

Once you have a class, you can define an object and use its methods. Here is how you might define a rocket and have it start to move up:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a Rocket object, and have it start to move up.
my_rocket = Rocket()
print("Rocket altitude:", my_rocket.y)

my_rocket.move_up()
print("Rocket altitude:", my_rocket.y)

my_rocket.move_up()
print("Rocket altitude:", my_rocket.y)
```

**Output:**

```
Rocket altitude: 0
Rocket altitude: 1
Rocket altitude: 2
```

To access an object's variables or methods, you give the name of the object and then use *dot notation* to access the variables and methods. So to get the y-value of *my_rocket*, you use *my_rocket.y*. To use the move_up() method on my_rocket, you write *my_rocket.move_up()*.

Once you have a class defined, you can create as many objects from that class as you want. Each object is its own instance of that class, with its own separate variables. All of the objects are capable of the same behavior, but each object's particular actions do not affect any of the other objects. Here is how you might make a simple fleet of rockets:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a fleet of 5 rockets, and store them in a list.
my_rockets = []
for x in range(0,5):
    new_rocket = Rocket()
    my_rockets.append(new_rocket)

# Show that each rocket is a separate object.
for rocket in my_rockets:
    print(rocket)
```

**Output:**

```
<__main__.Rocket object at 0x7f6f5073cd10>
<__main__.Rocket object at 0x7f6f5073cc90>
<__main__.Rocket object at 0x7f6f5073cbd0>
<__main__.Rocket object at 0x7f6f5077e410>
<__main__.Rocket object at 0x7f6f5077ead0>
```

You can see that each rocket is at a separate place in memory. By the way, if you understand list comprehensions, you can make the fleet of rockets in one line:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
```

```
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a fleet of 5 rockets, and store them in a list.
my_rockets = [Rocket() for x in range(0,5)]

# Show that each rocket is a separate object.
for rocket in my_rockets:
    print(rocket)
```

**Output:**

```
<__main__.Rocket object at 0x7f6f50789190> <__main__.Rocket object at 0x7f6f50763450> <__
main__.Rocket object at 0x7f6f507634d0> <__main__.Rocket object at 0x7f6f50763510> <__mai
n__.Rocket object at 0x7f6f50763550>
```

You can prove that each rocket has its own x and y values by moving just one of the rockets:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a fleet of 5 rockets, and store them in a list.
my_rockets = [Rocket() for x in range(0,5)]

# Move the first rocket up.
my_rockets[0].move_up()

# Show that only the first rocket has moved.
for rocket in my_rockets:
    print("Rocket altitude:", rocket.y)
```

**Output:**

```
Rocket altitude: 1
Rocket altitude: 0
Rocket altitude: 0
Rocket altitude: 0
Rocket altitude: 0
```

The syntax for classes may not be very clear at this point, but consider for a moment how you might create a rocket without using classes. You might store the x and y values in a dictionary, but you would have to write a lot of ugly, hard-to-maintain code to manage even a small set of rockets. As more features become incorporated into the Rocket class, you will see how much more efficiently real-world objects can be modeled with classes than they could be using just lists and dictionaries.

# Classes in Python 2.7

When you write a class in Python 2.7, you should always include the word `object` in parentheses when you define the class. This makes sure your Python 2.7 classes act like Python 3 classes, which will be helpful as your projects grow more complicated.

The simple version of the rocket class would look like this in Python 2.7:

```
class Rocket(object):
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
        def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
```

This syntax will work in Python 3 as well.

# Exercises

- Write a Python class named Rectangle constructed by a length and width and a method which will compute the area of a rectangle
- Write a Python class which has two methods get_String and print_String. get_String accept a string from the user and print_String print the string in upper case

# General terminology

A **class** is a body of code that defines the **attributes** and **behaviors** required to accurately model something you need for your program. You can model something from the real world, such as a rocket ship or a guitar string, or you can model something from a virtual world such as a rocket in a game, or a set of physical laws for a game engine.

An **attribute** is a piece of information. In code, an attribute is just a variable that is part of a class.

A **behavior** is an action that is defined within a class. These are made up of **methods**, which are just functions that are defined for the class.

An **object** is a particular instance of a class. An object has a certain set of values for all of the attributes (variables) in the class. You can have as many objects as you want for any one class.

There is much more to know, but these words will help you get started. They will make more sense as you see more examples, and start to use classes on your own.

# A closer look at the Rocket class

Now that you have seen a simple example of a class, and have learned some basic OOP terminology, it will be helpful to take a closer look at the Rocket class.

## The __init__() method

Here is the initial code block that defined the Rocket class:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
```

The first line shows how a class is created in Python. The keyword **class** tells Python that you are about to define a class. The rules for naming a class are the same rules you learned about naming variables, but there is a strong convention among Python programmers that classes should be named using CamelCase. If you are unfamiliar with CamelCase, it is a convention where each letter that starts a word is capitalized, with no underscores in the name.

The name of the class is followed by a set of parentheses. These parentheses will be empty for now, but later they may contain a class upon which the new class is based.

It is good practice to write a comment at the beginning of your class

Function names that start and end with two underscores are special built-in functions that Python uses in certain ways. The __init()__ method is one of these special functions. It is called automatically when you create an object from your class. The __init()__ method lets you make sure that all relevant attributes are set to their proper values when an object is created from the class, before the object is used. In this case, The __init__() method initializes the x and y values of the Rocket to 0.

The **self** keyword often takes people a little while to understand. The word "self" refers to the current object that you are working with. When you are writing a class, it lets you refer to certain attributes from any other part of the class. Basically, all methods in a class need the *self* object as their first argument, so they can access any attribute that is part of the class.

Now let's take a closer look at a **method**.

# A simple method

Here is the method that was defined for the Rocket class:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0

    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1
```

A method is just a function that is part of a class. Since it is just a function, you can do anything with a method that you learned about with functions. You can accept positional arguments, keyword arguments, an arbitrary list of argument values, an arbitrary dictionary of arguments, or any combination of these. Your arguments can return a value or a set of values if you want, or they can just do some work without returning any values.

Each method has to accept one argument by default, the value **self**. This is a reference to the particular object that is calling the method. This *self* argument gives you access to the calling object's attributes. In this example, the self argument is used to access a Rocket object's y-value. That value is increased by 1, every time the method move_up() is called by a particular Rocket object. This is probably still somewhat confusing, but it should start to make sense as you work through your own examples.

If you take a second look at what happens when a method is called, things might make a little more sense:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a Rocket object, and have it start to move up.
my_rocket = Rocket()
print("Rocket altitude:", my_rocket.y)

my_rocket.move_up()
print("Rocket altitude:", my_rocket.y)

my_rocket.move_up()
print("Rocket altitude:", my_rocket.y)
```

**Output:**

```
Rocket altitude: 0
Rocket altitude: 1
Rocket altitude: 2
```

In this example, a Rocket object is created and stored in the variable my_rocket. After this object is created, its y value is printed. The value of the attribute *y* is accessed using dot notation. The phrase *my_rocket.y* asks Python to return "the value of the variable y attached to the object my_rocket".

After the object my_rocket is created and its initial y-value is printed, the method move_up() is called. This tells Python to apply the method move_up() to the object my_rocket. Python finds the y-value associated with my_rocket and adds 1 to that value. This process is repeated several times, and you can see from the output that the y-value is in fact increasing.

# Making multiple objects from a class

One of the goals of object-oriented programming is to create reusable code. Once you have written the code for a class, you can create as many objects from that class as you need. It is worth mentioning at this point that classes are usually saved in a separate file, and then imported into the program you are working on. So you can build a library of classes, and use those classes over and over again in different programs. Once you know a class works well, you can leave it alone and know that the objects you create in a new program are going to work as they always have.

You can see this "code reusability" already when the Rocket class is used to make more than one Rocket object. Here is the code that made a fleet of Rocket objects:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a fleet of 5 rockets, and store them in a list.
my_rockets = []
for x in range(0,5):
    new_rocket = Rocket()
    my_rockets.append(new_rocket)

# Show that each rocket is a separate object.
for rocket in my_rockets:
    print(rocket)
```

**Output:**

```
<__main__.Rocket object at 0x7f6f50763090>
<__main__.Rocket object at 0x7f6f5077ead0>
<__main__.Rocket object at 0x7f6f50763990>
<__main__.Rocket object at 0x7f6f50763a10>
<__main__.Rocket object at 0x7f6f50763a50>
```

I will use the slightly longer approach of declaring an empty list, and then using a for loop to fill that list. That can be done slightly more efficiently than the previous example, by eliminating the temporary variable *new_rocket*:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Create a fleet of 5 rockets, and store them in a list.
my_rockets = []
for x in range(0,5):
    my_rockets.append(Rocket())

 # Show that each rocket is a separate object.
for rocket in my_rockets:
    print(rocket)
```

**Output:**

```
<__main__.Rocket object at 0x7f6f50763990>
<__main__.Rocket object at 0x7f6f50763a10>
<__main__.Rocket object at 0x7f6f50763750>
<__main__.Rocket object at 0x7f6f506fd8d0>
<__main__.Rocket object at 0x7f6f506fd6d0>
```

What exactly happens in this for loop? The line *my_rockets.append(Rocket())* is executed 5 times. Each time, a new Rocket object is created and then added to the list my_rockets. The __init__() method is executed once for each of these objects, so each object gets its own x and y value. When a method is called on one of these objects, the *self* variable allows access to just that object's attributes, and ensures that modifying one object does not affect any of the other objecs that have been created from the class.

Each of these objects can be worked with individually. At this point we are ready to move on and see how to add more functionality to the Rocket class. We will work slowly, and give you the chance to start writing your own simple classes.

# A quick check-in

If all of this makes sense, then the rest of your work with classes will involve learning a lot of details about how classes can be used in more flexible and powerful ways. If this does not make any sense, you could try a few different things:

- Reread the previous sections, and see if things start to make any more sense.
- Type out these examples in your own editor, and run them. Try making some changes, and see what happens.
- Try the next exercise, and see if it helps solidify some of the concepts you have been reading about.
- Read on. The next sections are going to add more functionality to the Rocket class. These steps will involve rehashing some of what has already been covered, in a slightly different way.

Classes are a huge topic, and once you understand them you will probably use them for the rest of your life as a programmer. If you are brand new to this, be patient and trust that things will start to sink in.

# Exercises

### *Your Own Rocket*

- Without looking back at the previous examples, try to recreate the Rocket class as it has been shown so far.
  - Define the Rocket() class.
  - Define the __init__() method, which sets an x and a y value for each Rocket object.
  - Define the move_up() method.
  - Create a Rocket object.
  - Print the object.
  - Print the object's y-value.
  - Move the rocket up, and print its y-value again.
  - Create a fleet of rockets, and prove that they are indeed separate Rocket objects.

# Refining the Rocket class

The Rocket class so far is very simple. It can be made a little more interesting with some refinements to the __init__() method, and by the addition of some methods.

## Accepting parameters for the __init__() method

The __init__() method is run automatically one time when you create a new object from a class. The __init__() method for the Rocket class so far is pretty simple:

```
class Rocket():
     # Rocket simulates a rocket ship for a game,
     #  or a physics simulation.
    def __init__(self):
        # Each rocket has an (x,y) position.
        self.x = 0
        self.y = 0
    def move_up(self):
         # Increment the y-position of the rocket.
         self.y += 1
```

All the __init__() method does so far is set the x and y values for the rocket to 0. We can easily add a couple keyword arguments so that new rockets can be initialized at any position:

```
class Rocket():
     # Rocket simulates a rocket ship for a game,
     #  or a physics simulation.
    def __init__(self, x=0, y=0):
        # Each rocket has an (x,y) position.
        self.x = x
        self.y = y
    def move_up(self):
         # Increment the y-position of the rocket.
         self.y += 1
```

Now when you create a new Rocket object you have the choice of passing in arbitrary initial values for x and y:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self, x=0, y=0):
        # Each rocket has an (x,y) position.
        self.x = x
        self.y = y
    def move_up(self):
        # Increment the y-position of the rocket.
        self.y += 1

# Make a series of rockets at different starting places.
rockets = []
rockets.append(Rocket())
rockets.append(Rocket(0,10))
rockets.append(Rocket(100,0))

# Show where each rocket is.
for index, rocket in enumerate(rockets):
    print("Rocket %d is at (%d, %d)." % (index, rocket.x, rocket.y))
```

**Output:**

```
Rocket 0 is at (0, 0).
Rocket 1 is at (0, 10).
Rocket 2 is at (100, 0).
```

# Accepting parameters in a method

The __init__ method is just a special method that serves a particular purpose, which is to help create new objects from a class. Any method in a class can accept parameters of any kind. With this in mind, the move_up() method can be made much more flexible. By accepting keyword arguments, the move_up() method can be rewritten as a more general move_rocket() method. This new method will allow the rocket to be moved any amount, in any direction:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self, x=0, y=0):
        # Each rocket has an (x,y) position.
        self.x = x
        self.y = y
```

```
    def move_rocket(self, x_increment=0, y_increment=1):
        # Move the rocket according to the paremeters given.
        #  Default behavior is to move the rocket up one unit.
        self.x += x_increment
        self.y += y_increment
```

The paremeters for the move() method are named x_increment and y_increment rather than x and y. It's good to emphasize that these are changes in the x and y position, not new values for the actual position of the rocket. By carefully choosing the right default values, we can define a meaningful default behavior. If someone calls the method move_rocket() with no parameters, the rocket will simply move up one unit in the y-direciton. Note that this method can be given negative values to move the rocket left or right:

```
class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self, x=0, y=0):
        # Each rocket has an (x,y) position.
        self.x = x
        self.y = y
    def move_rocket(self, x_increment=0, y_increment=1):
        # Move the rocket according to the paremeters given.
        #  Default behavior is to move the rocket up one unit.
        self.x += x_increment
        self.y += y_increment

# Create three rockets.
rockets = [Rocket() for x in range(0,3)]

# Move each rocket a different amount.
rockets[0].move_rocket()
rockets[1].move_rocket(10,10)
rockets[2].move_rocket(-10,0)

# Show where each rocket is.
for index, rocket in enumerate(rockets):
    print("Rocket %d is at (%d, %d)." % (index, rocket.x, rocket.y))
```

**Output:**

```
Rocket 0 is at (0, 1).
Rocket 1 is at (10, 10).
Rocket 2 is at (-10, 0).
```

# Adding a new method

One of the strengths of object-oriented programming is the ability to closely model real-world phenomena by adding appropriate attributes and behaviors to classes. One of the jobs of a team piloting a rocket is to make sure the rocket does not get too close to any other rockets. Let's add a method that will report the distance from one rocket to any other rocket.

If you are not familiar with distance calculations, there is a fairly simple formula to tell the distance between two points if you know the x and y values of each point. This new method performs that calculation, and then returns the resulting distance.

```
from math import sqrt  class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self, x=0, y=0):
        # Each rocket has an (x,y) position.
        self.x = x
        self.y = y
    def move_rocket(self, x_increment=0, y_increment=1):
        # Move the rocket according to the paremeters given.
        #  Default behavior is to move the rocket up one unit.
        self.x += x_increment
        self.y += y_increment
    def get_distance(self, other_rocket):
        # Calculates the distance from this rocket to another rocket,
        #  and returns that value.
        distance = sqrt((self.x-other_rocket.x)**2+(self.y-other_rocket.y)**2)
        return distance

# Make two rockets, at different places.
rocket_0 = Rocket()
rocket_1 = Rocket(10,5)

# Show the distance between them.
distance = rocket_0.get_distance(rocket_1)
print("The rockets are %f units apart." % distance)
```

**Output:**

```
The rockets are 11.180340 units apart.
```

Hopefully these short refinements show that you can extend a class' attributes and behavior to model the phenomena you are interested in as closely as you want. The rocket could have a name, a crew capacity, a payload, a certain amount of fuel, and any number of other attributes. You could define any behavior you want for the rocket, including interactions with other rockets and launch facilities, gravitational fields, and whatever you

need it to! There are techniques for managing these more complex interactions, but what you have just seen is the core of object-oriented programming.

At this point you should try your hand at writing some classes of your own. After trying some exercises, we will look at object inheritance, and then you will be ready to move on for now.

# Exercises

## *Your Own Rocket 2*

- There are enough new concepts here that you might want to try re-creating the Rocket class as it has been developed so far, looking at the examples as little as possible. Once you have your own version, regardless of how much you needed to look at the example, you can modify the class and explore the possibilities of what you have already learned.
  - Re-create the Rocket class as it has been developed so far:
    - Define the Rocket() class.
    - Define the __init__() method. Let your __init__() method accept x and y values for the initial position of the rocket. Make sure the default behavior is to position the rocket at (0,0).
    - Define the move_rocket() method. The method should accept an amount to move left or right, and an amount to move up or down.
    - Create a Rocket object. Move the rocket around, printing its position after each move.
    - Create a small fleet of rockets. Move several of them around, and print their final positions to prove that each rocket can move independently of the other rockets.
    - Define the get_distance() method. The method should accept a Rocket object, and calculate the distance between the current rocket and the rocket that is passed into the method.
    - Use the get_distance() method to print the distances between several of the rockets in your fleet.

## *Rocket Attributes*

- Start with a copy of the Rocket class, either one you made from a previous exercise or the latest version from the last section.
- Add several of your own attributes to the __init__() function. The values of your attributes can be set automatically by the __init__ function, or they can be set by paremeters passed into __init__().
- Create a rocket and print the values for the attributes you have created, to show they have been set correctly.
- Create a small fleet of rockets, and set different values for one of the attributes you have created. Print the values of these attributes for each rocket in your fleet, to show that they have been set properly for each rocket.

- If you are not sure what kind of attributes to add, you could consider storing the height of the rocket, the crew size, the name of the rocket, the speed of the rocket, or many other possible characteristics of a rocket.

## *Rocket Methods*

- Start with a copy of the Rocket class, either one you made from a previous exercise or the latest version from the last section.
- Add a new method to the class. This is probably a little more challenging than adding attributes, but give it a try.
  - Think of what rockets do, and make a very simple version of that behavior using print statements. For example, rockets lift off when they are launched. You could make a method called *launch()*, and all it would do is print a statement such as "The rocket has lifted off!" If your rocket has a name, this sentence could be more descriptive.
  - You could make a very simple *land_rocket()* method that simply sets the x and y values of the rocket back to 0. Print the position before and after calling the *land_rocket()* method to make sure your method is doing what it's supposed to.
  - If you enjoy working with math, you could implement a *safety_check()* method. This method would take in another rocket object, and call the get_distance() method on that rocket. Then it would check if that rocket is too close, and print a warning message if the rocket is too close. If there is zero distance between the two rockets, your method could print a message such as, "The rockets have crashed!" (Be careful; getting a zero distance could mean that you accidentally found the distance between a rocket and itself, rather than a second rocket.)

## *Person Class*

- Modeling a person is a classic exercise for people who are trying to learn how to write classes. We are all familiar with characteristics and behaviors of people, so it is a good exercise to try.
  - Define a Person() class.
  - In the __init()__ function, define several attributes of a person. Good attributes to consider are name, age, place of birth, and anything else you like to know about the people in your life.
  - Write one method. This could be as simple as *introduce_yourself()*. This method would print out a statement such as, "Hello, my name is Eric."
  - You could also make a method such as *age_person()*. A simple version of this method would just add 1 to the person's age.
    - A more complicated version of this method would involve storing the person's birthdate rather than their age, and then calculating the age whenever the age is requested. But dealing with dates and times is not particularly easy if you've never done it in any other programming language before.
  - Create a person, set the attribute values appropriately, and print out information about the person.
  - Call your method on the person you created. Make sure your method executed properly; if the method does not print anything out directly, print something before and after calling the method to make sure it did what it was supposed to.

*Car Class*

- Modeling a car is another classic exercise.
  - Define a Car() class.
  - In the __init__() function, define several attributes of a car. Some good attributes to consider are make (Subaru, Audi, Volvo...), model (Outback, allroad, C30), year, num_doors, owner, or any other aspect of a car you care to include in your class.
  - Write one method. This could be something such as *describe_car()*. This method could print a series of statements that describe the car, using the information that is stored in the attributes. You could also write a method that adjusts the mileage of the car or tracks its position.
  - Create a car object, and use your method.
  - Create several car objects with different values for the attributes. Use your method on several of your cars.

# Inheritance

One of the most important goals of the object-oriented approach to programming is the creation of stable, reliable, reusable code. If you had to create a new class for every kind of object you wanted to model, you would hardly have any reusable code. In Python and any other language that supports OOP, one class can **inherit** from another class. This means you can base a new class on an existing class; the new class *inherits* all of the attributes and behavior of the class it is based on. A new class can override any undesirable attributes or behavior of the class it inherits from, and it can add any new attributes or behavior that are appropriate. The original class is called the **parent** class, and the new class is a **child** of the parent class. The parent class is also called a **superclass**, and the child class is also called a **subclass**.

The child class inherits all attributes and behavior from the parent class, but any attributes that are defined in the child class are not available to the parent class. This may be obvious to many people, but it is worth stating. This also means a child class can override behavior of the parent class. If a child class defines a method that also appears in the parent class, objects of the child class will use the new method rather than the parent class method.

To better understand inheritance, let's look at an example of a class that can be based on the Rocket class.

# The SpaceShuttle class

If you wanted to model a space shuttle, you could write an entirely new class. But a space shuttle is just a special kind of rocket. Instead of writing an entirely new class, you can inherit all of the attributes and behavior of a Rocket, and then add a few appropriate attributes and behavior for a Shuttle.

One of the most significant characteristics of a space shuttle is that it can be reused. So the only difference we will add at this point is to record the number of flights the shutttle has completed. Everything else you need to know about a shuttle has already been coded into the Rocket class.

Here is what the Shuttle class looks like:

```
from math import sqrt  class Rocket():
     # Rocket simulates a rocket ship for a game,
     #  or a physics simulation.
     def __init__(self, x=0, y=0):
         # Each rocket has an (x,y) position.
         self.x = x
         self.y = y
     def move_rocket(self, x_increment=0, y_increment=1):
         # Move the rocket according to the paremeters given.
         #  Default behavior is to move the rocket up one unit.
         self.x += x_increment
         self.y += y_increment
     def get_distance(self, other_rocket):
         # Calculates the distance from this rocket to another rocket,
         #  and returns that value.
         distance = sqrt((self.x-other_rocket.x)**2+(self.y-other_rocket.y)**2)
         return distance

class Shuttle(Rocket):
    # Shuttle simulates a space shuttle, which is really
    #  just a reusable rocket.

    def __init__(self, x=0, y=0, flights_completed=0):
        super().__init__(x, y)
        self.flights_completed = flights_completed

shuttle = Shuttle(10,0,3)
print(shuttle)
```

```
<__main__.Shuttle object at 0x7f1e62ba6cd0>
```

When a new class is based on an existing class, you write the name of the parent class in parentheses when you define the new class:

```
class NewClass(ParentClass):
```

The __init__() function of the new class needs to call the __init__() function of the parent class. The __init__() function of the new class needs to accept all of the parameters required to build an object from the parent class, and these parameters need to be passed to the __init__() function of the parent class. The *super().__init__()* function takes care of this:

```
class NewClass(ParentClass):
    def __init__(self, arguments_new_class, arguments_parent_class):
        super().__init__(arguments_parent_class)
        # Code for initializing an object of the new class.
```

The *super()* function passes the *self* argument to the parent class automatically. You could also do this by explicitly naming the parent class when you call the __init__() function, but you then have to include the *self* argument manually:

```
class Shuttle(Rocket):
     # Shuttle simulates a space shuttle, which is really
     #  just a reusable rocket.
     def __init__(self, x=0, y=0, flights_completed=0):
        Rocket.__init__(self, x, y)
        self.flights_completed = flights_completed
```

This might seem a little easier to read, but it is preferable to use the *super()* syntax. When you use *super()*, you don't need to explicitly name the parent class, so your code is more resilient to later changes. As you learn more about classes, you will be able to write child classes that inherit from multiple parent classes, and the *super()* function will call the parent classes' __init__() functions for you, in one line. This explicit approach to calling the parent class' __init__() function is included so that you will be less confused if you see it in someone else's code.

The output above shows that a new Shuttle object was created. This new Shuttle object can store the number of flights completed, but it also has all of the functionality of the Rocket class: it has a position that can be changed, and it can calculate the distance between itself and other rockets or shuttles. This can be demonstrated by creating several rockets and shuttles, and then finding the distance between one shuttle and all the other shuttles and rockets. This example uses a simple function called randint, which generates a random integer between a lower and upper bound, to determine the position of each rocket and shuttle:

```
from math import sqrt
from random import randint
 class Rocket():
     # Rocket simulates a rocket ship for a game,
     #  or a physics simulation.
     def __init__(self, x=0, y=0):
         # Each rocket has an (x,y) position.
         self.x = x
         self.y = y
     def move_rocket(self, x_increment=0, y_increment=1):
         # Move the rocket according to the paremeters given.
         #  Default behavior is to move the rocket up one unit.
         self.x += x_increment
         self.y += y_increment
     def get_distance(self, other_rocket):
         # Calculates the distance from this rocket to another rocket,
         #  and returns that value.
         distance = sqrt((self.x-other_rocket.x)**2+(self.y-other_rocket.y)**2)
         return distance
```

```
class Shuttle(Rocket):
     # Shuttle simulates a space shuttle, which is really
     #  just a reusable rocket.
     def __init__(self, x=0, y=0, flights_completed=0):
         super().__init__(x, y)
         self.flights_completed = flights_completed

# Create several shuttles and rockets, with random positions.
#  Shuttles have a random number of flights completed.
shuttles = []
for x in range(0,3):
    x = randint(0,100)
    y = randint(1,100)
    flights_completed = randint(0,10)
    shuttles.append(Shuttle(x, y, flights_completed))

rockets = []
for x in range(0,3):
    x = randint(0,100)
    y = randint(1,100)
    rockets.append(Rocket(x, y))

# Show the number of flights completed for each shuttle.
for index, shuttle in enumerate(shuttles):
    print("Shuttle %d has completed %d flights." % (index, shuttle.flights_completed))

print("\n")
# Show the distance from the first shuttle to all other shuttles.
first_shuttle = shuttles[0]
for index, shuttle in enumerate(shuttles):
    distance = first_shuttle.get_distance(shuttle)
    print("The first shuttle is %f units away from shuttle %d." % (distance, index))

print("\n")
# Show the distance from the first shuttle to all other rockets.
for index, rocket in enumerate(rockets):
    distance = first_shuttle.get_distance(rocket)
    print("The first shuttle is %f units away from rocket %d." % (distance, index))
```

**Output:**

```
Shuttle 0 has completed 7 flights.
Shuttle 1 has completed 10 flights.
Shuttle 2 has completed 7 flights.
```

```
The first shuttle is 0.000000 units away from shuttle 0.
The first shuttle is 25.806976 units away from shuttle 1.
The first shuttle is 40.706265 units away from shuttle 2.

The first shuttle is 25.079872 units away from rocket 0.
The first shuttle is 60.415230 units away from rocket 1.
The first shuttle is 35.468296 units away from rocket 2.
```

Inheritance is a powerful feature of object-oriented programming. Using just what you have seen so far about classes, you can model an incredible variety of real-world and virtual phenomena with a high degree of accuracy. The code you write has the potential to be stable and reusable in a variety of applications.

# Inheritance in Python 2.7

The *super()* method has a slightly different syntax in Python 2.7:

```
class NewClass(ParentClass):
        def __init__(self, arguments_new_class, arguments_parent_class):
        super(NewClass, self).__init__(arguments_parent_class)
        # Code for initializing an object of the new class.
```

Notice that you have to explicitly pass the arguments *NewClass* and *self* when you call *super()* in Python 2.7. The SpaceShuttle class would look like this:

```
from math import sqrt  class Rocket(object):
     # Rocket simulates a rocket ship for a game,
     #  or a physics simulation.
     def __init__(self, x=0, y=0):
         # Each rocket has an (x,y) position.
         self.x = x
         self.y = y
     def move_rocket(self, x_increment=0, y_increment=1):
         # Move the rocket according to the paremeters given.
         #  Default behavior is to move the rocket up one unit.
         self.x += x_increment
         self.y += y_increment
     def get_distance(self, other_rocket):
         # Calculates the distance from this rocket to another rocket,
         #  and returns that value.
         distance = sqrt((self.x-other_rocket.x)**2+(self.y-other_rocket.y)**2)
         return distance
```

```
class Shuttle(Rocket):
    # Shuttle simulates a space shuttle, which is really
    #  just a reusable rocket.

    def __init__(self, x=0, y=0, flights_completed=0):
        super(Shuttle, self).__init__(x, y)
        self.flights_completed = flights_completed

shuttle = Shuttle(10,0,3)
print(shuttle)
```

**Output:**

```
<__main__.Shuttle object at 0x7f1e60080810>
```

This syntax works in Python 3 as well.

# Exercises

### *Student Class*

- Start with your program from Person Class.
- Make a new class called Student that inherits from Person.
- Define some attributes that a student has, which other people don't have.
  - A student has a school they are associated with, a graduation year, a gpa, and other particular attributes.
- Create a Student object, and prove that you have used inheritance correctly.
  - Set some attribute values for the student, that are only coded in the Person class.
  - Set some attribute values for the student, that are only coded in the Student class.
  - Print the values for all of these attributes.

### *Refining Shuttle*

- Take the latest version of the Shuttle class. Extend it.
  - Add more attributes that are particular to shuttles such as maximum number of flights, capability of supporting spacewalks, and capability of docking with the ISS.
  - Add one more method to the class, that relates to shuttle behavior. This method could simply print a statement, such as "Docking with the ISS," for a dock_ISS() method.
  - Prove that your refinements work by creating a Shuttle object with these attributes, and then call your new method.
  -

# Modules and classes

Now that you are starting to work with classes, your files are going to grow longer. This is good, because it means your programs are probably doing more interesting things. But it is bad, because longer files can be more difficult to work with. Python allows you to save your classes in another file and then import them into the program you are working on. This has the added advantage of isolating your classes into files that can be used in any number of different programs. As you use your classes repeatedly, the classes become more reliable and complete overall.

## Storing a single class in a module

When you save a class into a separate file, that file is called a **module**. You can have any number of classes in a single module. There are a number of ways you can then import the class you are interested in.

Start out by saving just the Rocket class into a file called *rocket.py*. Notice the naming convention being used here: the module is saved with a lowercase name, and the class starts with an uppercase letter. This convention is pretty important for a number of reasons, and it is a really good idea to follow the convention.

```
# Save as rocket.py
from math import sqrt  class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self, x=0, y=0):
        # Each rocket has an (x,y) position.
        self.x = x
        self.y = y
    def move_rocket(self, x_increment=0, y_increment=1):
        # Move the rocket according to the paremeters given.
        #  Default behavior is to move the rocket up one unit.
        self.x += x_increment
        self.y += y_increment
    def get_distance(self, other_rocket):
        # Calculates the distance from this rocket to another rocket,
        #  and returns that value.
        distance = sqrt((self.x-other_rocket.x)**2+(self.y-other_rocket.y)**2)
        return distance
```

Make a separate file called *rocket_game.py*. If you are more interested in science than games, feel free to call this file something like *rocket_simulation.py*. Again, to use standard naming conventions, make sure you are using a lowercase_underscore name for this file.

```
# Save as rocket_game.py
from rocket import Rocket  rocket = Rocket()
 print("The rocket is at (%d, %d)." % (rocket.x, rocket.y))
```

```
Output: The rocket is at (0, 0).
```

This is a really clean and uncluttered file. A rocket is now something you can define in your programs, without the details of the rocket's implementation cluttering up your file. You don't have to include all the class code for a rocket in each of your files that deals with rockets; the code defining rocket attributes and behavior lives in one file, and can be used anywhere.

The first line tells Python to look for a file called *rocket.py*. It looks for that file in the same directory as your current program. You can put your classes in other directories, but we will get to that convention a bit later. Notice that you do not

When Python finds the file *rocket.py*, it looks for a class called *Rocket*. When it finds that class, it imports that code into the current file, without you ever seeing that code. You are then free to use the class Rocket as you have seen it used in previous examples.

# Storing multiple classes in a module

A module is simply a file that contains one or more classes or functions, so the Shuttle class actually belongs in the rocket module as well:

```
# Save as rocket.py from math import sqrt  class Rocket():
    # Rocket simulates a rocket ship for a game,
    #  or a physics simulation.
    def __init__(self, x=0, y=0):
        # Each rocket has an (x,y) position.
        self.x = x
        self.y = y
    def move_rocket(self, x_increment=0, y_increment=1):
        # Move the rocket according to the paremeters given.
        #  Default behavior is to move the rocket up one unit.
        self.x += x_increment
        self.y += y_increment
    def get_distance(self, other_rocket):
        # Calculates the distance from this rocket to another rocket,
        #  and returns that value.
        distance = sqrt((self.x-other_rocket.x)**2+(self.y-other_rocket.y)**2)
        return distance

class Shuttle(Rocket):
    # Shuttle simulates a space shuttle, which is really
    #  just a reusable rocket.

    def __init__(self, x=0, y=0, flights_completed=0):
        super().__init__(x, y)
        self.flights_completed = flights_completed
```

Now you can import the Rocket and the Shuttle class, and use them both in a clean uncluttered program file:

```
# Save as rocket_game.py
from rocket import Rocket, Shuttle
 rocket = Rocket()
 print("The rocket is at (%d, %d)." % (rocket.x, rocket.y))
shuttle = Shuttle()
print("\nThe shuttle is at (%d, %d)." % (shuttle.x, shuttle.y))
print("The shuttle has completed %d flights." % shuttle.flights_completed)
```

**Output:**

```
The rocket is at (0, 0).
The shuttle is at (0, 0).
The shuttle has completed 0 flights.
```

The first line tells Python to import both the *Rocket* and the *Shuttle* classes from the *rocket* module. You don't have to import every class in a module; you can pick and choose the classes you care to use, and Python will only spend time processing those particular classes.

# A number of ways to import modules and classes

There are several ways to import modules and classes, and each has its own merits.

## import *module_name*

The syntax for importing classes that was just shown:

```
from module_name import ClassName
```

is straightforward, and is used quite commonly. It allows you to use the class names directly in your program, so you have very clean and readable code. This can be a problem, however, if the names of the classes you are importing conflict with names that have already been used in the program you are working on. This is unlikely to happen in the short programs you have been seeing here, but if you were working on a larger program it is quite possible that the class you want to import from someone else's work would happen to have a name you have already used in your program. In this case, you can use simply import the module itself:

```
# Save as rocket_game.py
import rocket  rocket_0 = rocket.Rocket()
print("The rocket is at (%d, %d)." % (rocket_0.x, rocket_0.y))
shuttle_0 = rocket.Shuttle()
print("\nThe shuttle is at (%d, %d)." % (shuttle_0.x, shuttle_0.y))
print("The shuttle has completed %d flights." % shuttle_0.flights_completed)
```

```
Output:
The rocket is at (0, 0).
The shuttle is at (0, 0).
The shuttle has completed 0 flights.
```

The general syntax for this kind of import is:

```
import module_name
```

After this, classes are accessed using dot notation:

```
module_name.ClassName
```

This prevents some name conflicts. If you were reading carefully however, you might have noticed that the variable name *rocket* in the previous example had to be changed because it has the same name as the module itself. This is not good, because in a longer program that could mean a lot of renaming.

## import *module_name* as *local_module_name*

There is another syntax for imports that is quite useful:

```
import module_name as local_module_name
```

When you are importing a module into one of your projects, you are free to choose any name you want for the module in your project. So the last example could be rewritten in a way that the variable name *rocket* would not need to be changed:

```
# Save as rocket_game.py
import rocket as rocket_module
rocket = rocket_module.Rocket()
print("The rocket is at (%d, %d)." % (rocket.x, rocket.y))
shuttle = rocket_module.Shuttle()
print("\nThe shuttle is at (%d, %d)." % (shuttle.x, shuttle.y))
print("The shuttle has completed %d flights." % shuttle.flights_completed)
```

```
Output:
The rocket is at (0, 0).
The shuttle is at (0, 0).
The shuttle has completed 0 flights.
```

This approach is often used to shorten the name of the module, so you don't have to type a long module name before each class name that you want to use. But it is easy to shorten a name so much that you force people reading your code to scroll to the top of your file and see what the shortened name stands for. In this example,

```
import rocket as rocket_module
```

leads to much more readable code than something like:

```
import rocket as r
```

## from *module_name* import *

There is one more import syntax that you should be aware of, but you should probably avoid using. This syntax imports all of the available classes and functions in a module:

```
from module_name import *
```

This is not recommended, for a couple reasons. First of all, you may have no idea what all the names of the classes and functions in a module are. If you accidentally give one of your variables the same name as a name from the module, you will have naming conflicts. Also, you may be importing way more code into your program than you need.

If you really need all the functions and classes from a module, just import the module and use the `module_name.ClassName` syntax in your program.
You will get a sense of how to write your imports as you read more Python code, and as you write and share some of your own code.

# A module of functions

You can use modules to store a set of functions you want available in different programs as well, even if those functions are not attached to any one class. To do this, you save the functions into a file, and then import that file just as you saw in the last section. Here is a really simple example; save this is *multiplying.py*:

```
# Save as multiplying.py
def double(x):
     return 2*x

def triple(x):
     return 3*x

def quadruple(x):
     return 4*x
```

Now you can import the file *multiplying.py*, and use these functions. Using the `from module_name import function_name` syntax:

```
from multiplying import double, triple, quadruple
 print(double(5))
 print(triple(5))
 print(quadruple(5))

Output:
10
15
20
```

Using the `import module_name` syntax:

```
import multiplying
 print(multiplying.double(5))
 print(multiplying.triple(5))
 print(multiplying.quadruple(5))
```

```
Output:
10
15
20
```

Using the `import module_name as local_module_name` syntax:

```
import multiplying as m
 print(m.double(5))
 print(m.triple(5))
 print(m.quadruple(5))
```

```
Output:
10
15
20
```

Using the `from module_name import *` syntax:

```
from multiplying import *
 print(double(5))
 print(triple(5))
 print(quadruple(5))
```

```
Output:
10
15
20
```

# Exercises

### *Importing Student*

- Take your program from Student Class
  - Save your Person and Student classes in a separate file called *person.py*.
  - Save the code that uses these classes in four separate files.
    - In the first file, use the `from module_name import ClassName` syntax to make your program run.
    - In the second file, use the `import module_name` syntax.
    - In the third file, use the `import module_name as different_local_module_name` syntax.
    - In the fourth file, use the `import *` syntax.

### *Importing Car*

- Take your program from Car Class
  - Save your Car class in a separate file called *car.py*.
  - Save the code that uses the car class into four separate files.
    - In the first file, use the `from module_name import ClassName` syntax to make your program run.
    - In the second file, use the `import module_name` syntax.
    - In the third file, use the `import module_name as different_local_module_name` syntax.
    - In the fourth file, use the `import *` syntax.

# Challenges:

### *Passgen:*

Security is so important nowadays but it still seems like nobody has strong passwords! That's where you come in. Write a program that gives users to ability to select a password strength and length. Then generate a random password for them. The program should save the users password after it has been generated and be able to accommodate new users. Bonus points if the generation is cryptographically secure. If you are able to securely store the passwords as well you win a prize!!

Your Program Should:

- Use a class or classes
- Use methods

Motivate to Innovate...

STEM BUILDERS Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS

contactus@stembuilders.com

www.stembuilders.com